

Homework — 2.1.2 Thinking Ahead — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] Completed table:

Item	Input / Output / Precondition
The member's ID scanned at the desk	Input (data flowing into the routine)
The updated loan record written to the database	Output (a result flowing out)
"The member has no unpaid fines"	Precondition (must be true before issuing goes ahead)
The book's barcode scanned at the desk	Input
The printed receipt showing the due date	Output
"The book is currently on loan to this member"	Precondition (must be true before the return routine runs — otherwise a book could be marked returned that was never borrowed, corrupting the loan records)

Mark: 2 correct rows per mark (6 rows → 3 marks). The bracketed reasons are not required.

2. [2] Correct ordering:

Step	Order
The computed timetable is stored in the cache.	4
A user requests the London → Manchester timetable.	1
The system checks the cache — the timetable is not there (a cache miss).	2
A second user requests the same timetable and is served the stored copy straight from the cache.	6
The timetable is computed by querying the live database.	3
The timetable is returned to the first user.	5

Full sequence: request → cache checked (miss) → computed from the database → stored in the cache → returned to the user → next request served from the cache. Mark: all six positions correct = 2; four or five correct = 1.

3. (a) [1] Any one: the expensive computation/database query happens only **once** per day, so pages load faster for every user and server load is greatly reduced.

(b) [2] 1 mark: the cached copy becomes **stale** — if a train is delayed or cancelled during the day, the cache still holds the midnight version. 1 mark: every user is then shown **out-of-date/wrong** times for the

rest of the day because the design never invalidates or refreshes the cached data.

(c) [1] Any one: refresh/invalidate the cache at shorter intervals (e.g. every few minutes); invalidate the cached timetable whenever a delay/cancellation is recorded so it is recomputed; cache only the fixed timetable but fetch live delay information separately.

4. [2] Any two (1 each): faster development — write once, drop into all three games; fewer bugs because the component is already **tested**; consistency — all games save/load the same way; easier maintenance — fix or improve it once and all three benefit.

5. [3] 1 mark each, applied to the order routine: **input** — e.g. items chosen, quantities, delivery address, payment details; **precondition** — e.g. the basket must be non-empty / the items must be in stock / the user logged in *before* payment is taken; **cache** — e.g. the menu/prices, or the user's saved delivery address, stored for fast reuse. Must be applied to the scenario; generic definitions score poorly.

Part B (6 marks)

6. [2] 1 mark keep: e.g. student ID, name, year/class, the subjects/sets they take. 1 mark discard: e.g. eye colour, favourite food, home address detail — irrelevant to scheduling lessons. (*Discard need not be justified here; one keep + one discard.*)

7. [2] 1 mark: an interrupt is a signal sent to the processor that an event needs attention, causing it to pause the current task. 1 mark, any one reason: to respond promptly to I/O (key press, printer ready), errors, or hardware/timer events without constantly polling; the OS saves state, services the interrupt (via the ISR), then resumes.

8. [2] Any two (1 each): clock speed; number of cores; cache size/amount of cache; word length/bus width; pipelining. (*Accept any two valid factors.*)

Total: 14 + 6 = 20 marks